CENG 201 – Task 3 Report

Topic: Discharge Stack Implementation using Linked List

Time Complexity Analysis:

In this task, a stack data structure was implemented using a linked list to manage discharge records in a hospital system. The stack uses a single pointer called top, which points to the topmost node in the stack.

The push method adds a new discharge record to the top of the stack. The operation involves creating a new node, setting its next pointer to the current top, and updating the top pointer to point to the new node. Since we only need to update pointers and do not need to traverse the list, the time complexity of this operation is O(1).

The pop method removes and returns the discharge record from the top of the stack. The operation involves checking if the stack is empty, retrieving the data from the top node, and updating the top pointer to the next node. Since we directly access the top node without traversal, the time complexity of this operation is O(1).

The peek method returns the discharge record at the top of the stack without removing it. This operation simply accesses the top node's data, resulting in O(1) time complexity.

Stack vs. Queue Comparison:

A stack follows the LIFO (Last In First Out) principle, where the last element added is the first one to be removed. This is suitable for discharge records where we may need to access the most recently discharged patient first. A queue, on the other hand, follows FIFO (First In First Out), which would process the oldest discharge record first.

In this hospital management scenario, where we may need to quickly access the most recent discharge information, a stack is the appropriate data structure choice. The

constant time operations for push, pop, and peek make it efficient for managing discharge records.